

BARAKA'IT SOLUTION

SAE 2.03 – Supervision Réseau

GUIDE D'UTILISATION

Logiciel de supervision réseau – Version 1.0

Projet	SAE 2.03 – Mettre en place une solution informatique
Entreprise	Baraka'IT Solution
Formation	BUT Réseaux & Télécommunications – 1ère année
Version	1.0 – Mai 2026
Auteurs	GALANT Adame · DILMAHAMOD Rehaan · BOUCHRANI Ambdouroihamane · SAUTRON-CICIA Markus

1. Introduction

Ce document constitue le guide d'utilisation du logiciel de supervision réseau développé dans le cadre de la SAE 2.03 par le groupe 2 de Baraka'IT Solution.

Il est destiné à tout administrateur réseau souhaitant déployer et utiliser l'application pour surveiller l'état des équipements d'un réseau local.

1.1 Objectif du logiciel

Le logiciel permet de :

- surveiller en temps réel la disponibilité des équipements réseau via ICMP (ping)
- vérifier l'accessibilité de services web via HTTP/HTTPS
- détecter automatiquement les anomalies et afficher un tableau de bord clair
- conserver un historique horodaté de chaque vérification au format JSON
- déclencher des alertes visuelles et écrire dans un journal des événements

1.2 Statuts possibles

Le logiciel utilise exclusivement deux statuts : OK (équipement joignable) et ANOMALIE (équipement injoignable ou service inaccessible). Aucun autre libellé ne doit apparaître.

Statut	Signification	Couleur d'affichage
OK	Équipement joignable et opérationnel	Vert –  UP
ANOMALIE	Équipement injoignable ou service inaccessible	Rouge –  ANOMALIE

2. Prérequis

2.1 Système d'exploitation

Système	Version minimale	Compatibilité
Debian / Ubuntu	Debian 11 / Ubuntu 20.04	✓ Compatible
macOS	macOS 12 Monterey	✓ Compatible
Windows	Windows 10 / 11	✓ Compatible

2.2 Vérifier la version de Python

Avant toute installation, vérifier que Python 3.8 ou supérieur est bien installé sur la machine.

Vérification de la version installée

```
python3 --version
```

La commande doit retourner une version supérieure ou égale à 3.8, par exemple :

```
Python 3.11.2
```

⚠ Si la commande retourne *command not found* ou une version inférieure à 3.8, Python doit être installé ou mis à jour avant de continuer.

Installer Python si absent ou version insuffisante

Sous Debian / Ubuntu :

```
sudo apt update
sudo apt install python3 python3-pip python3-venv -y
python3 --version # Vérifier après installation
```

Sous macOS :

Télécharger l'installateur officiel sur : <https://www.python.org/downloads/>

Ou via Homebrew si installé :

```
brew install python3
```

Sous Windows :

Télécharger l'installateur sur : <https://www.python.org/downloads/windows/>

⚠ Sous Windows, cocher impérativement l'option *Add Python to PATH* lors de l'installation, sinon les commandes *python* et *pip* ne seront pas reconnues dans le terminal.

2.3 Dépendances logicielles

Composant	Version	Installation	Rôle
Python 3	3.8 ou supérieur	Voir section 2.2	Langage principal
pip	21+	apt install python3-pip	Gestionnaire de paquets
requests	2.31.0	pip install requests	Vérification HTTP
customtkinter	5.2.2	pip install customtkinter	Interface graphique
Git	2.x	apt install git	Clonage du dépôt

 *Sur Linux, l'exécution du ping ICMP nécessite des droits root (sudo). L'utilisation de `sudo python3 main.py` est obligatoire pour que la supervision ICMP fonctionne correctement.*

3. Installation

3.1 Cloner le dépôt GitHub

Le dépôt est public — aucun compte GitHub ni clé SSH n'est nécessaire pour cloner le projet. Utiliser simplement la commande HTTPS ci-dessous :

```
git clone https://github.com/galantadame-del/SAE203_Groupe2.git
cd SAE203_Groupe2
```

i Si Git n'est pas installé : `sudo apt install git -y` (Linux) ou télécharger sur <https://git-scm.com/downloads> (macOS/Windows).

3.2 Créer l'environnement virtuel Python

Un environnement virtuel isole les dépendances du projet du reste du système.

```
python3 -m venv venv
source venv/bin/activate          # Linux / macOS
venv\Scripts\activate            # Windows
```

Si venv n'est pas installé, l'installer avec cette commande :

```
sudo apt install python3-venv
```

i L'invite de commande doit afficher (venv) avant le nom d'utilisateur pour confirmer que l'environnement virtuel est actif.

3.3 Installer les dépendances

```
pip install -r requirements.txt
```

Contenu du fichier requirements.txt :

```
requests==2.31.0
customtkinter==5.2.2
```

3.4 Créer les répertoires nécessaires

```
mkdir -p data logs
touch logs/.gitkeep
```

3.5 Vérifier la structure du projet

Après installation, la structure du projet doit ressembler à ceci :

```
SAE203_Groupe2/
├─ main.py
├─ config.json
├─ requirements.txt
├─ .gitignore
├─ src/
│   ├─ __init__.py
│   ├─ collecte.py
│   ├─ analyse.py
│   ├─ alerte.py
│   ├─ affichage.py
│   ├─ stockage.py
│   └─ config_manager.py
└─ data/
    ├─ cibles.json
    └─ historique.json (créé automatiquement)
```

✓ Si tous ces fichiers sont présents, l'installation est correcte.

4. Configuration

4.1 Paramètres généraux — config.json

Le fichier config.json à la racine du projet permet de personnaliser le comportement global du superviseur sans modifier le code source.

```
{
  "intervalle_secondes": 30,
  "timeout_icmp_secondes": 2,
  "timeout_http_secondes": 5,
  "mode_debug": false,
  "mode_une_seule_fois": false
}
```

Paramètre	Type	Valeur par défaut	Description
intervalle_secondes	entier	30	Fréquence des vérifications en secondes
timeout_icmp_secondes	entier	2	Délai max avant ANOMALIE pour un ping
timeout_http_secondes	entier	5	Délai max avant ANOMALIE pour HTTP
mode_debug	booléen	false	Affiche les logs détaillés dans le terminal
mode_une_seule_fois	booléen	false	Exécute un seul cycle puis s'arrête (debug)

4.2 Équipements à superviser — data/cibles.json

Le fichier data/cibles.json définit la liste des équipements à surveiller. Il est généré automatiquement au premier démarrage par le scanner réseau, ou peut être édité manuellement.

```
{
  "cibles": [
    {
      "nom": "Serveur Web",
      "adresse": "192.168.1.10",
      "type": "ping"
    },
    {
      "nom": "Portail Intranet",
      "adresse": "https://192.168.1.10",
      "type": "http"
    }
  ]
}
```

Champ	Valeurs acceptées	Description
nom	Texte libre	Nom affiché dans le tableau de bord
adresse	IP ou URL	Adresse IPv4 ou URL complète (avec https://)
type	"ping" ou "http"	Protocole de vérification à utiliser

 Pour les vérifications HTTP, l'adresse doit commencer par http:// ou https://. Pour les vérifications ping, indiquer uniquement l'adresse IP.

5. Utilisation

5.1 Démarrage du logiciel

5.1.1 Sous Linux / Debian (recommandé)

Ouvrir un terminal, se placer dans le répertoire du projet, activer le venv, puis lancer :

```
cd SAE203_Groupe2
source venv/bin/activate
sudo venv/bin/python3 main.py
```

 *Utiliser `sudo venv/bin/python3 main.py` et non `sudo python3 main.py`. Sans ça, `sudo` utilise le Python système sans les dépendances installées dans le venv.*

5.1.2 Sous macOS

```
cd SAE203_Groupe2
source venv/bin/activate
sudo python3 main.py
```

5.1.3 Sous Windows

```
cd SAE203_Groupe2
venv\Scripts\activate
python main.py
```

 *Sous Windows, les droits administrateur ne sont généralement pas requis pour le ping. Si des erreurs apparaissent, exécuter le terminal en tant qu'administrateur.*

5.2 Démarrage automatique — scan du réseau

Au premier lancement (si data/cibles.json est absent ou vide), le logiciel effectue automatiquement un scan du réseau local :

- L'IP locale de la machine est détectée automatiquement
- Toutes les IP du sous-réseau /24 sont testées en parallèle
- Les équipements joignables sont ajoutés dans data/cibles.json
- La supervision démarre immédiatement après le scan

Exemple de sortie au démarrage :

```
[SCANNER] IP locale détectée : 192.168.1.42
[SCANNER] Scan du réseau      : 192.168.1.0/24
[SCANNER] Scan en cours... (peut prendre 30-60 secondes)
[SCANNER] ✓ Trouvé : 192.168.1.1    → Routeur-1
[SCANNER] ✓ Trouvé : 192.168.1.10   → Serveur-10
[SCANNER] 2 équipement(s) détecté(s).
```

i Si cibles.json existe et contient déjà des cibles, le scan est ignoré. Pour forcer un nouveau scan, supprimer le fichier data/cibles.json avant le démarrage.

5.3 Lecture du tableau de bord

Une fois le logiciel lancé, un tableau de bord s'affiche dans le terminal et se rafraîchit automatiquement à chaque cycle de supervision.

Exemple d'affichage :

```

  SUPERVISEUR RÉSEAU - Baraka'IT Solution
  20/05/2026 à 10:32:15

  +-----+-----+-----+
  | Équipement | Adresse | Statut |
  +-----+-----+-----+
  | Routeur-1  | 192.168.1.1 | ✓ UP   |
  | Serveur Web | 192.168.1.10 | ✓ UP   |
  | Imprimante-RDC | 192.168.1.50 | ✗ ANOMALIE |
  +-----+-----+-----+
  | Résumé : ✓ OK : 2 | ✗ ANOMALIE : 1 |
  +-----+-----+-----+
  | ⚠ DERNIÈRE ANOMALIE : Imprimante-RDC - injoignable
  | Détectée le 20/05/2026 à 10:32:20
  +-----+-----+-----+
  | Prochain cycle dans : 28s [██████████░░░░░░] 45%
  +-----+-----+-----+

```

Élément	Description
En-tête	Nom du projet, date et heure du dernier rafraîchissement
Tableau des équipements	Liste des cibles avec adresse et statut (OK / ANOMALIE)
Résumé	Compteurs globaux : nombre d'OK et d'anomalies
Zone d'alerte	Mémoire et affiche la dernière anomalie détectée avec horodatage
Barre de progression	Indique le temps restant avant le prochain cycle de supervision

5.4 Arrêt du logiciel

Pour arrêter proprement le logiciel, appuyer sur :

```
Ctrl + C
```

Le logiciel affiche alors :

```
[INFO] Supervision arrêtée proprement.
```

 *Aucune perte de données — l'historique JSON est sauvegardé avant l'arrêt.*

6. Fichiers générés automatiquement

6.1 Historique des vérifications — data/historique.json

Ce fichier enregistre chaque résultat de vérification avec son horodatage. Il est mis à jour à chaque cycle.

```
{
  "historique": [
    {
      "nom": "Serveur Web",
      "adresse": "192.168.1.10",
      "type": "ping",
      "statut": "OK",
      "joignable": true,
      "message": "Hôte joignable",
      "timestamp": "2026-05-20T10:32:15"
    },
    {
      "nom": "Imprimante-RDC",
      "adresse": "192.168.1.50",
      "type": "ping",
      "statut": "ANOMALIE",
      "joignable": false,
      "message": "Hôte injoignable (timeout)",
      "timestamp": "2026-05-20T10:32:20"
    }
  ]
}
```

6.2 Journal des alertes — logs/alertes.log

Ce fichier texte enregistre toutes les anomalies détectées avec leur horodatage précis. Une ligne est ajoutée à chaque nouvelle anomalie.

```
2026-05-20 10:32:20 - ANOMALIE - Imprimante-RDC (192.168.1.50) : Hôte
injoignable
2026-05-20 10:33:50 - ANOMALIE - Serveur-Backup (192.168.1.20) : Timeout
HTTP
```

Pour consulter le journal en temps réel depuis un second terminal :

```
tail -f logs/alertes.log
```

7. Dépannage

7.1 Tableau des erreurs courantes

Erreur / Symptôme	Solution
ImportError: cannot import name 'analyser'	Remplacer src/analyse.py par la version corrigée (voir Section 3.5)
Permission denied pour le ping	Lancer avec sudo venv/bin/python3 main.py (Linux/macOS)
ModuleNotFoundError: No module named 'requests'	Vérifier que le venv est activé et relancer pip install -r requirements.txt
Aucun équipement détecté par le scanner	Vérifier la connexion réseau. Tester manuellement : ping 192.168.1.1
Authentication failed pour GitHub	Utiliser SSH (git clone git@github.com:...) plutôt que HTTPS
L'interface se fige	Un thread bloque – vérifier que threading.daemon = True est bien présent
historique.json : JSONDecodeError	Supprimer data/historique.json – il sera recréé automatiquement

7.2 Configurer SSH pour GitHub (premier démarrage)

Si le clonage demande un mot de passe, utiliser SSH :

1. Générer une clé SSH sur la machine :

```
ssh-keygen -t ed25519 -C "votre@email.com"
```

2. Afficher la clé publique :

```
cat ~/.ssh/id_ed25519.pub
```

3. Copier la clé et l'ajouter sur GitHub : Settings → SSH and GPG keys → New SSH key

4. Tester la connexion :

```
ssh -T git@github.com
```

5. Cloner avec SSH :

```
git clone git@github.com:galantadame-del/SAE203_Groupe2.git
```

7.3 Forcer un nouveau scan réseau

Pour rescanner le réseau (si de nouveaux équipements ont été ajoutés) :

```
rm data/cibles.json  
sudo venv/bin/python3 main.py
```

7.4 Mettre à jour le logiciel

```
cd SAE203_Groupe2
```

```
git pull origin main  
pip install -r requirements.txt  
sudo venv/bin/python3 main.py
```

8. Informations sur le projet

8.1 Équipe de développement

Nom	Rôle	Responsabilités
GALANT Adame	Chef de projet	Coordination, planning, communication, Git
DILMAHAMOD Rehaan	Développeur Front	Interface CustomTkinter, affichage, tableau de bord
BOUHRANI Ambdouroihamane	Développeur Back	Collecte ICMP/HTTP, analyse, alertes, stockage, scanner
SAUTRON-CICIA Markus	Testeur / Documentaliste	Tests unitaires, scénario démo, documentation

8.2 Dépôt GitHub

URL : https://github.com/galantadame-del/SAE203_Groupe2

8.3 Planning

Jalon	Date	Statut
Envoi PowerPoint	3 avril 2026	✓ Réalisé
Envoi document technique	13 avril 2026	✓ Réalisé
Démonstration technique	30 avril 2026	✓ Réalisé
Soutenance finale	20 mai 2026	📋 En cours